$$p_{bit} = \left(1 - \frac{1}{m}\right)^{k \cdot n} \approx e^{-\frac{k \cdot n}{m}}$$

To have a false positive, all the `k` bits corresponding to an element `v` must have been set to `1` independently, and the probability that all of those `k` bits are `1` is given by:

$$p\left(n, m, k\right) = \left(1 - p_{bit}\right)^k = \left(1 - \left(1 - \frac{1}{m}\right)^{k \cdot n}\right)^k \approx \left(1 - e^{-\frac{k \cdot n}{m}}\right)^k$$

Which is, lo and behold, the probability formula we gave at the beginning of this section.

At this stage, as we mentioned in the last section, we can consider `n` and `m` as constants: it makes sense because in many cases we know how many elements we need to add to the Bloom filter (`n`) and how much bits of storage we can afford (`m`); what we would like to do is then trading performance for accuracy by tuning `k`, the number of (universal) hash functions we use.

This is equivalent to finding the global minimum of function `f`, defined as:

$$f\left(k\right) = \left(1 - e^{-\frac{k \cdot n}{m}}\right)^k$$

If you know some calculus, you probably have already guessed that we need to compute the derivatives of f with